

Où et comment la loi gaussienne intervient dans les modèles (et pourquoi ce n'est pas un output du modèle)

DeepEdgeBenchmark

22 juillet 2026

Introduction

Ce document résume deux questions posées sur le calcul des intervalles de confiance et du CRPS (Continuous Ranked Probability Score) dans les modèles de prévision du projet : où la loi gaussienne est-elle codée, à quoi s'applique-t-elle exactement, quand est-elle appelée, et surtout : est-elle produite directement par le modèle, ou ajoutée après coup par le code ?

1 Où est codée la loi gaussienne, et quand est-elle appelée

Il faut distinguer deux usages complètement séparés dans le code.

1.1 Usage 1 : construire les intervalles de confiance à 95 % (IC95) et les prévisions ponctuelles

C'est l'usage actif, utilisé par tous les modèles pour produire RMSE, IC95 et largeur d'IC95 dans le dashboard. Il n'a pas été modifié par l'ajout du CRPS empirique.

- **ARIMA-GARCH** (`models/arima_model.py`)
 - `arima_model.py`, lignes 152–154 et 164–166 :
`arch_model(resid, vol="Garch", p=1, q=1, dist="normal", rescale=False)`

Hypothèse gaussienne sur les *innovations du GARCH*, utilisée pour estimer par maximum de vraisemblance la volatilité conditionnelle. Appelée au fit initial et à chaque refit (tous les 20 pas par défaut) dans `run_arima_garch`.

- `arima_model.py`, ligne 52 : `Z_95 = 1.96`, appliqué lignes 172–173 :

$$\text{lower} = \text{last_price} \cdot e^{\mu - 1.96 \sigma}, \quad \text{upper} = \text{last_price} \cdot e^{\mu + 1.96 \sigma}$$

Suppose le rendement à 1 pas $\sim \mathcal{N}(\mu, \sigma)$. Appelée à chaque pas du walk-forward (Gate 2, horizon D1) et dans `next_step_arima_garch`.

- `benchmarks/multi_horizon.py`, lignes 68–69 : même formule, réutilisée pour l'horizon D7 et la prévision live (multi-pas, variance GARCH cumulée).
- **LSTM** (`models/lstm_model.py`)
 - Lignes 186–187 :

$$\text{lower} = \text{preds} - 1.96 \sigma_{\text{train}}, \quad \text{upper} = \text{preds} + 1.96 \sigma_{\text{train}}$$

où σ_{train} est l'écart-type des résidus *sur l'ensemble d'entraînement*, calculé **une seule fois**. Appelée à la fin de chaque `run_lstm` (Gate 2, D1) et dans `next_step_lstm`.

- `benchmarks/multi_horizon.py`, ligne 212 : même écart-type, élargi en $\sigma\sqrt{h}$ pour l'horizon D7/live.

- **Naive** (`models/naive_model.py`) : ligne 55, $Z_{95} = 1.96$, appliqué lignes 130–131 : dernier prix $\pm 1.96\sigma$ (marche aléatoire). Hors scope du CRPS (non demandé, baseline triviale).
- **SARIMA** (`models/sarima_model.py`) : aucun 1.96 écrit dans notre code – l’hypothèse gaussienne est **implicite** dans `SARIMAX.get_forecast().conf_int()` (statsmodels), dont le filtre de Kalman suppose des innovations gaussiennes par construction de la classe. Appelée à chaque refit de `run_sarima` (Gate 2, D1) et dans `forecast_from_fitted_sarima` (D7/live).
- **Prophet** (`models/prophet_model.py`) : même chose, pas de 1.96 explicite – `yhat_lower/yhat_upper` proviennent du modèle d’incertitude interne de la librairie Prophet (bruit gaussien sur tendance/saisonnalité par défaut). Appelée à chaque `model.predict()` dans `run_prophet` et en D7/live.
- **TSDiff** (`models/tsdiff_model.py`) : la docstring (ligne 12) parle de prédire “the Gaussian noise” – attention, cela décrit le *bruit de diffusion* que le denoiser apprend à prédire pendant l’entraînement (formulation DDPM/DDIM standard), **pas** la loi de sortie. La distribution prédictive finale de TSDiff est explicitement *non gaussienne* (ligne 19 : “a genuine predictive distribution rather than a Gaussian residual band”) – c’est un nuage de trajectoires simulées, sans hypothèse de forme.

1.2 Usage 2 : un CRPS gaussien fermé, préexistant et séparé

- `honest_eval/metrics.py`, lignes 231–240 : fonction `crps_gaussian(mu, sigma, actual)` – formule fermée du CRPS pour une loi prédictive gaussienne $\mathcal{N}(\mu, \sigma)$ (Gneiting & Raftery, cas gaussien), via `scipy.stats.norm`.
- Utilisée uniquement dans `experiments/pooled_analysis.py` (script d’analyse hors-ligne), qui dérive un σ implicite à partir de la largeur de l’IC95 :

$$\sigma_{\text{implicite}} = \frac{y_{\text{upper}} - y_{\text{lower}}}{2 \times 1.96}$$

puis applique la formule fermée gaussienne.

- **Jamais appelée** dans `model_artifacts/` ni `validation/` – donc jamais dans le pipeline quotidien ni le dashboard. C’est un CRPS *approximé* par hypothèse gaussienne, entièrement distinct du CRPS *empirique* (`crps_empirical`, Gneiting & Raftery éq. 20) calculé par `model_artifacts/crps_kpis.py` à partir des nuages bootstrap/Monte-Carlo-Dropout/TSDiff.

2 La gaussienne est-elle un output du modèle ?

Non. Dans aucun des cinq modèles la loi gaussienne n’est produite directement par le modèle – c’est toujours un choix de code ajouté autour, mais à deux niveaux très différents selon les cas.

2.1 LSTM et Naive : convention post-hoc, sans rapport avec le modèle

Le réseau LSTM se termine par une couche Dense(1) : il ne sort **qu’un seul nombre** (le prix prédit), aucune notion d’incertitude n’est produite par le réseau. La bande $\pm 1.96\sigma$ est construite entièrement à part, après l’entraînement :

```
train_preds = scaler.inverse_transform(model.predict(X_train)...).flatten()
std = np.std(train.values[seq_len:] - train_preds) # UN SEUL nombre
lower = preds - 1.96 * std
upper = preds + 1.96 * std
```

`std` est l'écart-type des erreurs du modèle sur le jeu d'entraînement, calculé **une seule fois**, puis recopié tel quel autour de **chaque** prédiction future – que le modèle soit en réalité plus ou moins confiant à cet instant précis n'entre jamais en jeu. C'est une convention manuelle, pas une information produite par le réseau. Le raisonnement est identique pour Naive.

2.2 ARIMA-GARCH, SARIMA, Prophet : l'hypothèse est dans le modèle statistique, pas dans un réseau de neurones

Ce ne sont pas des réseaux de neurones mais des modèles statistiques classiques, dont la procédure d'ajustement (maximum de vraisemblance) suppose *explicitement* un bruit gaussien pour estimer les paramètres.

- **GARCH** : le mot-clé `dist="normal"` est un choix explicite (on aurait pu écrire `dist="t"` pour une loi de Student, aux queues plus épaisses). Le GARCH estime bien un σ qui varie dans le temps (recalculé à chaque pas via `garch_res.forecast(...)`) – il y a donc un vrai calcul dynamique – mais la forme “gaussienne” de l'intervalle final reste un choix, pas un fait mesuré sur les données.
- **SARIMA** : SARIMAX de statsmodels utilise un filtre de Kalman dont la vraisemblance interne est gaussienne *par construction de la classe* (ce n'est même pas un paramètre réglable dans notre code) – `conf_int()` en ressort directement une borne gaussienne.
- **Prophet** : même principe, modèle bayésien interne à la librairie avec bruit d'observation gaussien par défaut.

Aucun de ces trois modèles n'est un réseau de neurones – ce sont des modèles paramétriques classiques où “gaussien” est une hypothèse de travail choisie au moment de l'ajustement, pas une propriété empirique des rendements observés. Or les rendements financiers sont connus pour avoir des queues plus épaisses qu'une loi normale (*fat tails*) – c'est précisément pour cette raison que le CRPS empirique (par bootstrap des résidus ou Monte Carlo Dropout) est utile : il ne fait aucune de ces hypothèses de forme.

2.3 TSDiff : le seul cas qui échappe complètement à la gaussienne

TSDiff ne sort ni un σ ni une hypothèse de forme : il tire un nuage de trajectoires (`sample_paths`) directement, sans jamais supposer une forme gaussienne pour la distribution prédictive finale. C'est exactement la raison pour laquelle il disposait déjà d'un CRPS empirique avant les autres modèles, et le point de départ de la demande d'étendre ce calcul aux quatre autres modèles.

En résumé

La gaussienne n'est jamais “vue” par le modèle sur les nouvelles données :

- pour **LSTM** et **Naive**, c'est un ruban de largeur fixe, recopié depuis l'erreur d'entraînement, indépendant de chaque nouvelle prédiction ;
- pour **ARIMA-GARCH**, **SARIMA** et **Prophet**, c'est une hypothèse de forme choisie pour pouvoir calculer analytiquement un intervalle à partir d'un σ estimé par le modèle statistique ;
- pour **TSDiff**, il n'y a aucune hypothèse de forme : la distribution prédictive est directement le nuage d'échantillons simulés.

3 Complément — fréquence d'appel, origine de la loi, et queues épaisses

Ajout du 24 juillet 2026, suite à trois questions posées sur ce document : la gaussienne est-elle appelée tous les jours de la validation ? Boucle-t-elle ? Vient-elle d'une librairie Python ou d'un calcul manuel ? Et pourquoi préférer une loi à queues épaisses ?

3.1 Une boucle quotidienne, mais pas homogène

La validation walk-forward est une boucle explicite : un pas par jour du jeu de validation. À chaque itération, le modèle est rappelé et produit une nouvelle prédiction accompagnée d'un nouvel intervalle. Ce qui est réellement recalculé chaque jour diffère cependant selon le modèle :

Modèle	Recalculé chaque jour	Recalculé seulement périodiquement / jamais
ARIMA-GARCH	μ et σ (intervalle complet)	Le réajustement complet du GARCH — tous les 20 jours seulement
SARIMA	Modèle entier + intervalle (conf_int)	—
Prophet	Modèle entier + intervalle	—
LSTM	Le centre (prédiction ponctuelle)	La largeur de la bande — calculée une seule fois sur les résidus d'entraînement
Naive	Le centre (dernier prix)	La largeur de la bande — calculée une seule fois sur les résidus d'entraînement

Autrement dit : la boucle quotidienne existe pour tous les modèles, mais seuls les trois modèles statistiques (ARIMA-GARCH, SARIMA, Prophet) remettent à jour la *largeur* de l'intervalle chaque jour ; pour LSTM et Naive, seul le centre bouge, la largeur reste figée sur toute la validation.

3.2 Origine de la loi : librairie statistique, ou calcul manuel ?

Les deux coexistent, et c'est une distinction importante :

Source	Origine réelle
ARIMA-GARCH	Librairie arch (dist="normal") — un ajustement par maximum de vraisemblance qui suppose explicitement des innovations gaussiennes.
SARIMA	Librairie statsmodels — le filtre de Kalman interne de la classe SARIMAX est gaussien par construction ; l'intervalle est calculé par la librairie elle-même.
Prophet	Modèle bayésien interne de la librairie prophet , bruit d'observation gaussien par défaut.
LSTM / Naive	Aucune librairie : calcul écrit à la main, prédiction $\pm 1,96 \times$ écart-type, où 1,96 est une constante recopiée (le 97,5 ^e centile d'une loi normale standard), pas calculée par un appel de librairie.
CRPS gaussien fermé (dashboard hebdo et analyses hors-ligne)	scipy.stats.norm est appelée directement (.cdf, .pdf) — la seule occurrence où la gaussienne est un objet mathématique de librairie réellement évalué, et non une simple constante recopiée.

Sur les cinq modèles, trois s'appuient donc sur une véritable librairie statistique qui « sait » ce qu'est une gaussienne (arch, statsmodels, prophet) ; les deux autres (LSTM, Naive) se contentent d'une formule manuelle avec un chiffre codé en dur.

3.3 Pourquoi une loi à queues épaisses plutôt qu'une gaussienne ?

La réponse est empirique : la calibration à trois niveaux mesurée sur le benchmark (cf. *Synthèse KPI probabilistes*, 21 juillet 2026) montre un motif systématique sur les modèles à intervalle gaussien :

Modèle	Centre 50% (cible 0,50)	Queues 95% (cible 0,95)
ARIMA-GARCH	0,57 — trop large	0,93 — trop étroit
SARIMA	0,61 — trop large	0,94 — trop étroit
LSTM	0,66 — trop large	0,94 — trop étroit
Naive	0,61 — trop large	0,94 — trop étroit

Au centre, l'intervalle couvre plus que promis (trop prudent). Aux extrêmes, il couvre moins que promis (trop confiant) : le vrai prix sort de la fourchette « sûre à 95 % » plus souvent que ce que le modèle annonce. Les mouvements extrêmes du marché (krachs, flambées) sont donc plus fréquents que ce qu'une gaussienne budgète — c'est la signature empirique des *queues épaisses* (excès de kurtosis) des rendements financiers.

Une gaussienne fait décroître la probabilité des événements extrêmes très vite (en $\exp(-x^2/2)$) : pour être plus généreuse aux extrêmes, il faudrait élargir tout l'intervalle uniformément, ce qui aggraverait encore le sur-couvrement déjà présent au centre. Une loi à queues épaisses (par exemple une loi de Student, que la librairie **arch** permet d'activer via `dist="t"` au lieu de `dist="normal"`) résout ce compromis : elle reste resserrée près du centre tout en réservant plus de probabilité aux mouvements rares — exactement ce qu'aucun des intervalles gaussiens utilisés ici n'arrive à faire simultanément.

3.4 Tableau complet, vérifié ligne par ligne (ajout du 29 juillet 2026)

Suite à la demande d'un tableau détaillé (rôle / librairie / fichier:ligne / en boucle ? / phase), sur le modèle de ce qui existe déjà pour le bruit de diffusion de TSDiff. Chaque ligne ci-dessous a été vérifiée dans le code du dépôt ou, pour les appels internes aux librairies (arch, statsmodels, prophet), directement dans le code source installé — pas seulement d'après la documentation de ces librairies.

Modèle	Rôle	Librairie / appel	Fichier : ligne	En boucle ?	Phase
LSTM	Perte d'entraînement — MSE = log-vraisemblance gaussienne implicite (σ non estimé)	loss="mse" (Keras)	lstm_model.py:127 (fit l.163-164)	Oui — epochs/batches internes à <code>model.fit()</code> (boucle Keras, non explicite en Python)	Training
LSTM	Intervalle 95% — pred $\pm$ 1,96·std(résidu train)	constante en dur	lstm_model.py:186-187 (aussi :224 next-step)	Non — vectorisé après la boucle walk-forward	Validation
ARIMA-GARCH	Loi d'innovation GARCH par défaut (dist='normal')	arch_model(..., dist=dist)	arima_model.py:208 (fit initial), :222 (refit)	Oui — refit tous les g arch_refit_freq pas (boucle l.215)	Training
ARIMA-GARCH	Quantile du PI ($\mu \pm \sigma \cdot q$) — sous dist='normal', q = stats.norm.ppf	Normal.ppf (lib. arch)	arch/univariate/distribution.py (lib.) via arima_model.py:131 _std_quantiles	Oui — même cadence que le refit GARCH	Validation
SARIMA	Intervalle de confiance — statsmodels utilise scipy.stats.norm.ppf par défaut (self.dist)	fc.conf_int(alpha=PI_ALPHA)	statsmodels/tsa/base/prediction.py (lib.) via sarima_model.py:117	Oui — refit + conf_int à chaque pas (boucle l.107)	Training + Validation
Prophet	Bruit d'observation gaussien du sampling postérieur	np.random.normal(0, sigma_obs, ...)	prophet/forecaster.py:1532 (sample_model, lib.) via prophet_model.py:151	Oui — 1000 tirages (uncertainty_samples par défaut) à chaque predict()	Validation
Naive	Intervalle de marche aléatoire gaussienne (prev $\pm$ 1,96 σ)	constante en dur	naive_model.py:138-139	Non — vectorisé	Validation
Naive	Nuage CRPS explicitement gaussien	rng.normal(0.0, sigma, size=...)	naive_model.py:144	Non — un seul tirage vectorisé (tous pas $\times$ échantillons)	Validation
Tous modèles	Constante Z95 utilisée dans le reporting	norm.ppf(0.975)	generate_distributions_dashboard.py:71	Non — calculée une fois au chargement du module	Reporting
Tous modèles + TSDiff	CRPS gaussien de reporting/sélection — σ rétro-déduit de la largeur du PI (hi-lo)/(2·Z95), réinjecté dans une loi normale	crps_gaussian(mu, sigma, actual)	honest_eval/metrics.py:231-240, via generate_distributions_dashboard.py:140-151, ensemble_weights.py:96, lstm_weekly_sweep.py:81, pooled_analysis.py:109	Oui — boucle sur (actif, modèle) / lignes selon l'appelant	Reporting / sélection de modèle
Tous modèles	Test de calibration PIT gaussien	stats.norm.cdf((y-mu)/sigma)	honest_eval/metrics.py:243-252, via honest_eval/targets.py:115	Non — vectorisé sur tout le jeu de test	Reporting / diagnostic

Lecture : « En boucle » distingue un recalcul à chaque pas du walk-forward d'un calcul unique réutilisé tel quel (cf. déjà §3.1). Les deux dernières lignes sont transversales aux 5 modèles (et à TSDiff pour la première) : elles vivent dans la couche d'évaluation/reporting, pas dans l'ajustement du modèle lui-même.

Correction — §1.2 (ajout du 29 juillet 2026)

Le §1.2 affirme que `crps_gaussian` n'est « jamais appelée dans `model_artifacts/` ». Ce n'est plus exact dans l'état actuel du code : `model_artifacts/generate_distributions_dashboard.py:151,204` l'appelle bien, pour chaque (actif, modèle) affiché dans `distributions_dashboard.html` (cf. *crps_gaussian_vs_empirique.pdf*, qui documente précisément cette lecture du dashboard). Le pipeline a donc évolué depuis la rédaction de ce document (22–24 juillet) — le CRPS gaussien fermé est aujourd'hui bien dans le chemin de reporting quotidien, pas seulement dans des scripts d'analyse hors-ligne.